



**RV College of  
Engineering®**

*Go, change the world*

## Unit -1

# Introduction to Programming

# Contents

1. Definition of a computer.
2. Components of computer system,
3. Programming Languages.
4. Design and implementation of efficient programs.
5. Program Design Tools: Algorithms,
6. Flowcharts and Pseudocodes.
7. Types of Errors

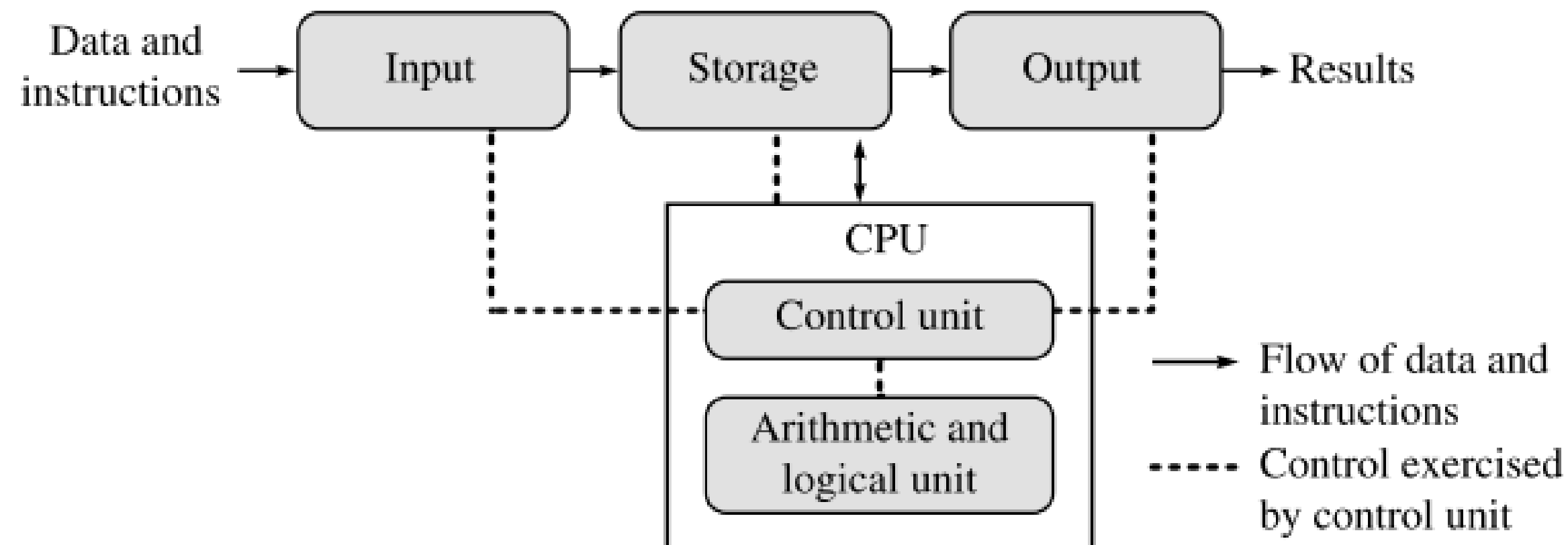
## 1. Definition of a computer:

A computer, in simple terms, can be defined as

“an electronic device that is designed to accept data, perform the required mathematical and logical operations, and produces reliable and expected result(output)”.



- A computer basically performs five major operations:
  1. Accepting data or instructions (input) : keyboard, mouse, scanner, trackball
  2. Storing data: Primary(RAM) and secondary memory(Disk Drives)
  3. Processing data: Control Unit(CU) and Arithmetic and Logic Unit(ALU)
  4. Displaying results (output): Monitor, printer, speakers etc.,
  5. Controlling and coordinating all above operations inside a computer



**COMPONENTS OF A COMPUTER SYSTEM:** The computer system consist of two components - Hardware and Software.

**1. Hardware:** Collection of Physical parts of the Computer that includes:

- **Memory**

- **Primary memory:**

RAM is essential for the smooth operation of computers and devices, providing temporary storage that enhances speed, supports multitasking, and facilitates real-time processing

- Random Access Memory (RAM: Volatile memory): types,

- Static RAM (SRAM), Dynamic RAM(DRAM)

- Read Only Memory non-volatile (ROM): Example - ROM-BIOS, types

- Rewritable ROM chips: Programmable ROM (PROM), Erasable PROM (EPROM), Electrically Erasable PROM(EEPROM) - camera, MP3 player etc.,

- Secondary memory: Hard disks (HDD/SSD), zipdrive, Pendrive, Flash drive etc., - formatted according to a file system that organizes the data into files and directories

- **Hard Disks(HDD/SSD)**

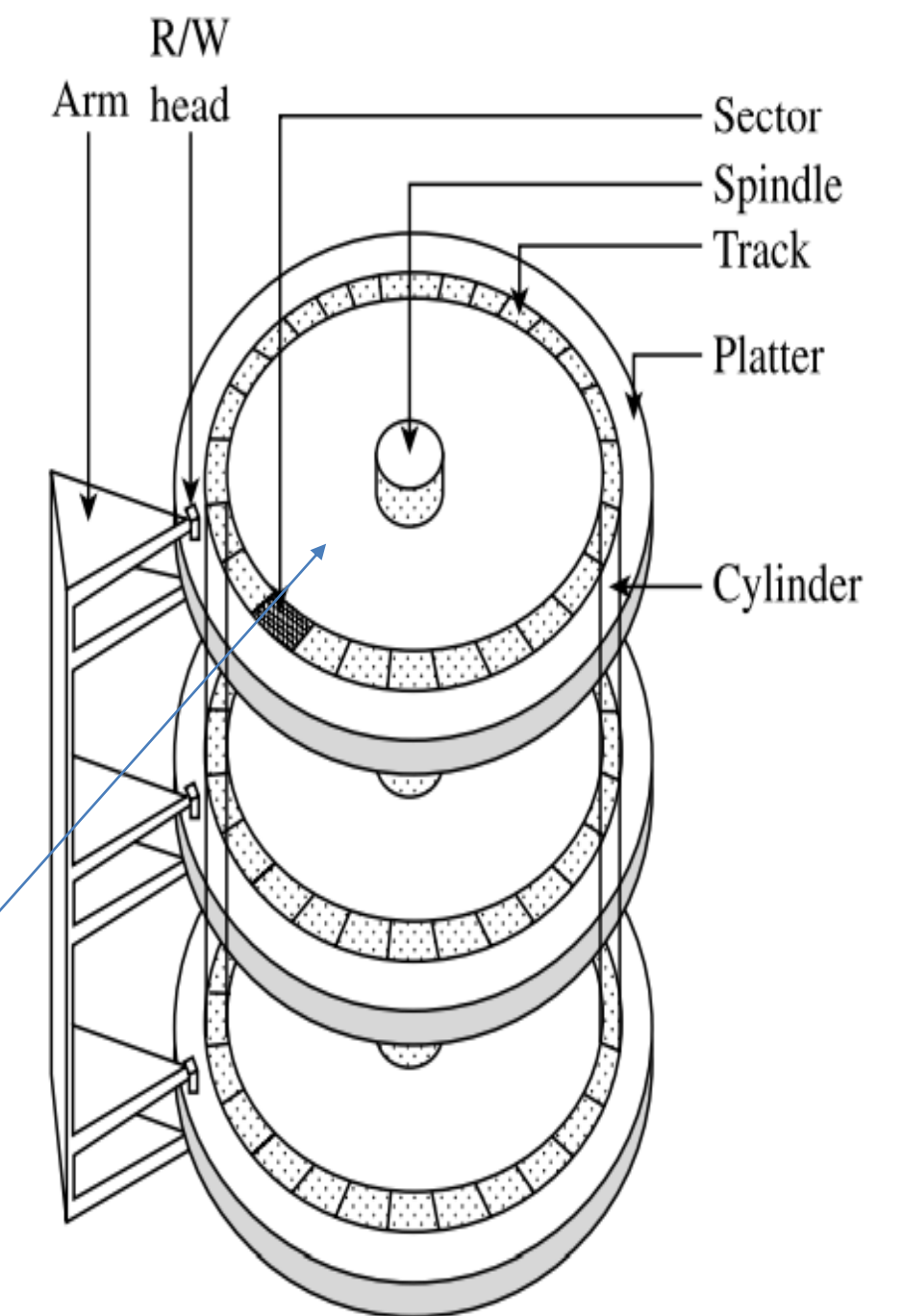
- **Processor**

- **Peripheral Devices/Input and Output Devices**

# Hard Disks

*Go, change the world*

- A hard disk is a set of disks(platters), stacked together like phonograph records, and each disk has data recorded electromagnetically in concentric circles known as **tracks**.
- Each platter requires two **read/write** (R/W) heads, one for each side.
- The R/W head move back and forth over the platter to read or write data
- Data is actually stored on the surface of a platter in **tracks** and **sectors**.

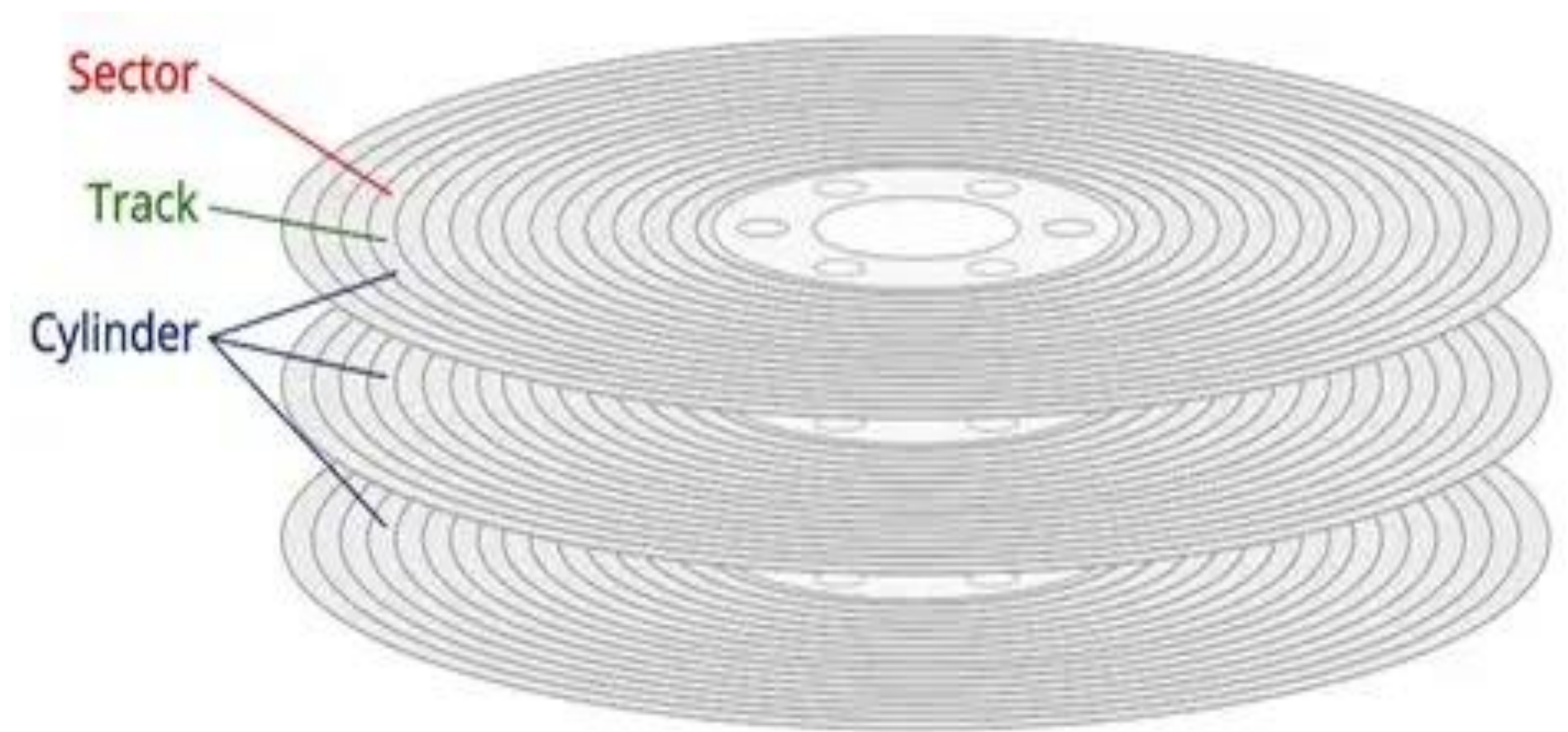


The performance of a hard disk depends on its access time. Access time is a combination of the following:

- **Seek time:** time taken to position the R/W head over the appropriate cylinder (~8 ms).
- **Rotational delay:** time taken to bring the target sector to rotate under the R/W head.
- **Transfer time:** time to transfer data from sector

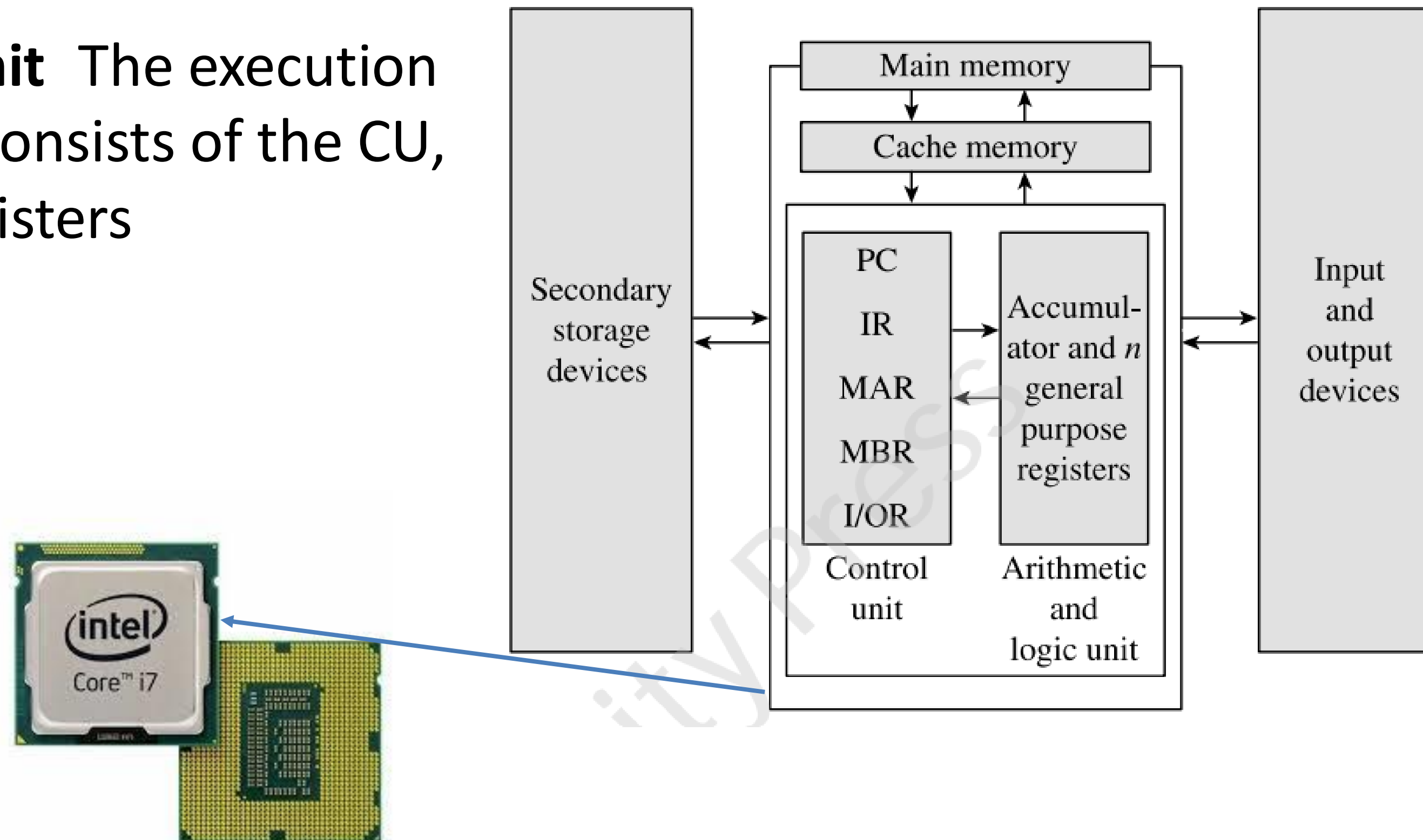
**Access time = seek time + rotational delay + transfer time.**

- **Disk latency:** time taken to initiate a transfer (seek time + rotational delay)



A basic processor consists of two main parts - **Execution Unit**, and a **Bus Interface Unit (BIU)**.

**Execution unit** The execution unit mainly consists of the CU, ALU, and registers



## **Control Unit (CU) - Main functions are,**

- guide and coordinate the computer operations.
- interprets the instructions (program) and initiates action to execute them.
- controls the flow of data through the computer system and directs (how they have to function) the ALU, registers, buses, and input/output (I/O) devices.
- Responsible for CPU Cycle (data fetching, encoding, and processing) and storing the results.

## **Arithmetic and logic unit:**

The ALU performs arithmetic (add, subtract, multiply, divide, etc.), comparison (less than, greater than, or equal to), and other operations.

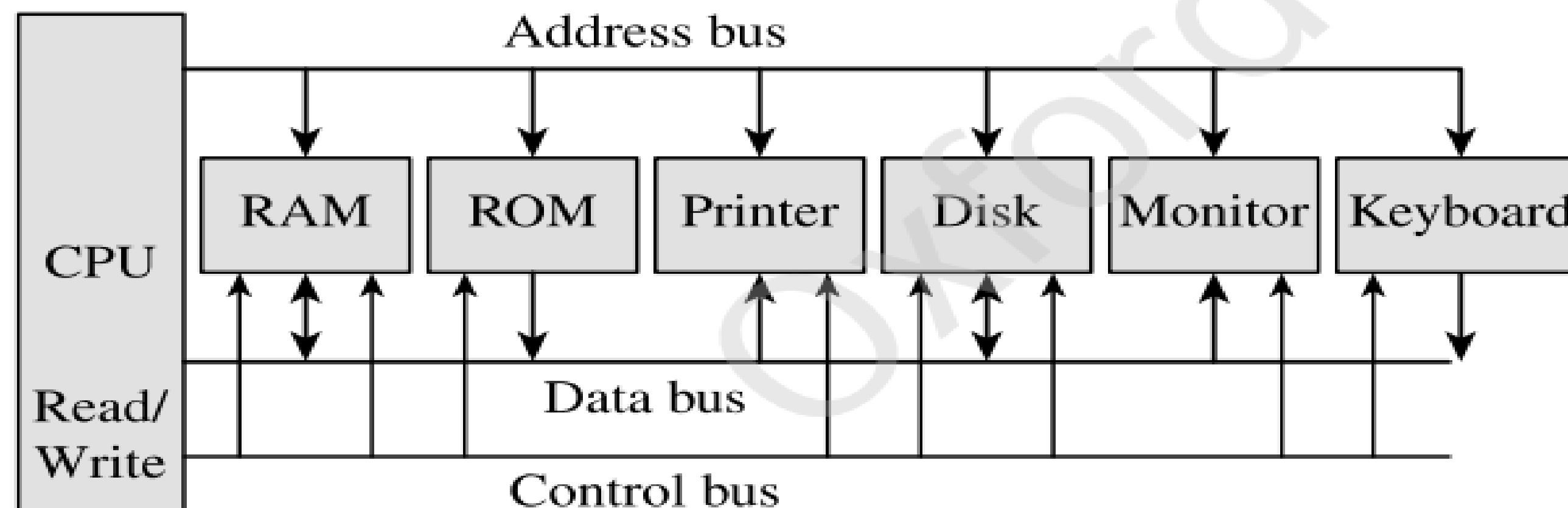
**Register(s):** is ultrafast computer memory that provides quick access to the data currently being used by ALU for processing and storing result.

Types of registers, each with a specific storage function:

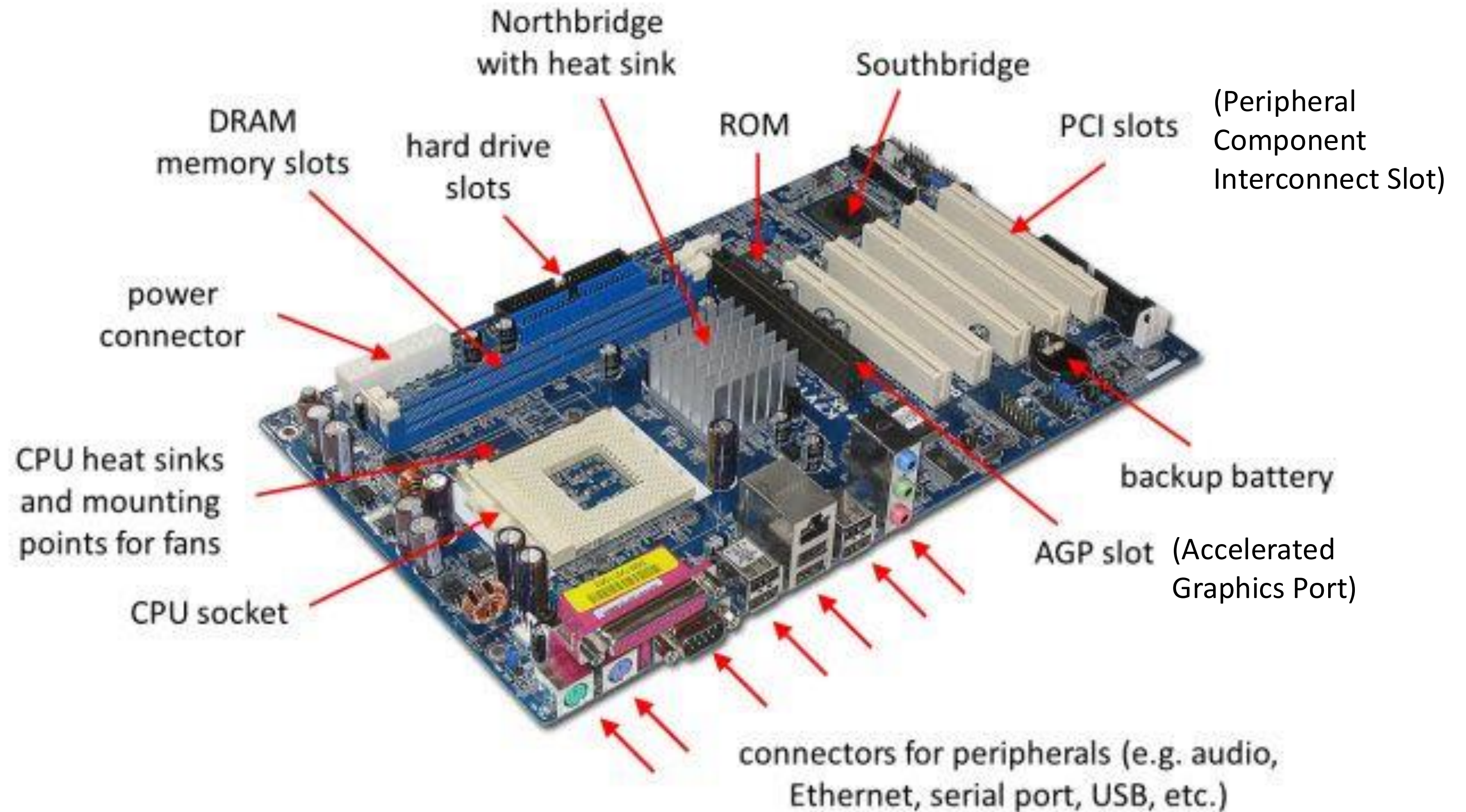
- **Accumulator and general-purpose registers:** to store the data brought from the main memory (RAM) and the intermediate results from ALU
- **Special purpose registers** – Memory Address Register (MAR), Memory Buffer Register (MBR), Instruction Register (IR), I/O register and Program Counter (PC)

**Bus Interface Unit:** provide functions for transferring data between the execution unit of the CPU and other components of the computer system that lie outside the CPU.

- There are three different types of busses - data bus, control bus, and address bus



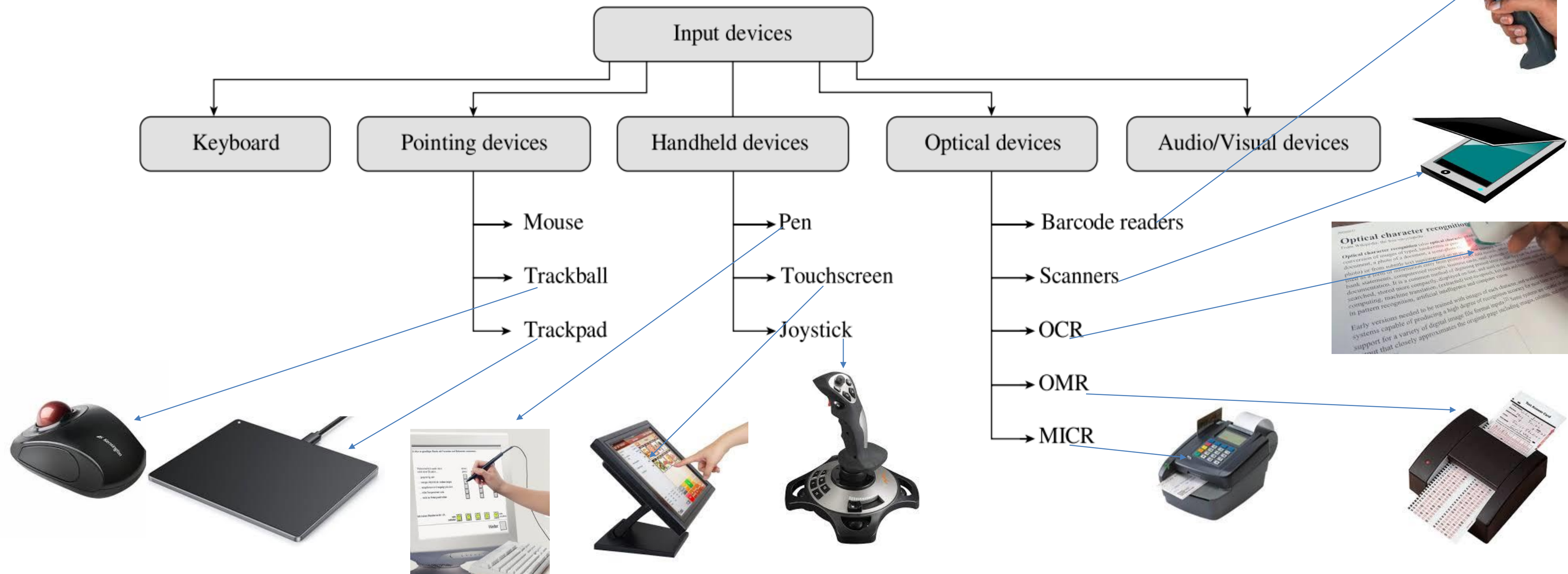
- The BIU puts the contents of the **program counter** on the **address bus**.
- Once the memory receives an address from the BIU, it places the contents at that address on the **data bus**, which is then transferred to the IR of the processor through the MBR.
- At this time, the contents of the program counter are modified (e.g., incremented by 1) so that it now stores the address of the next instruction



- **Instruction set** The instruction set is a set of commands that instructs the processor to perform specific tasks
- **System clock** A small quartz crystal circuit, that controls the timing of all operations within the computer system
- **Processor speed** is specified in MHz or GHz varies from type of computers
- **Pipelining and parallel processing** is a technique with which the processor can fetch the second instruction before completing the execution of the first instruction. (superscalar processors)

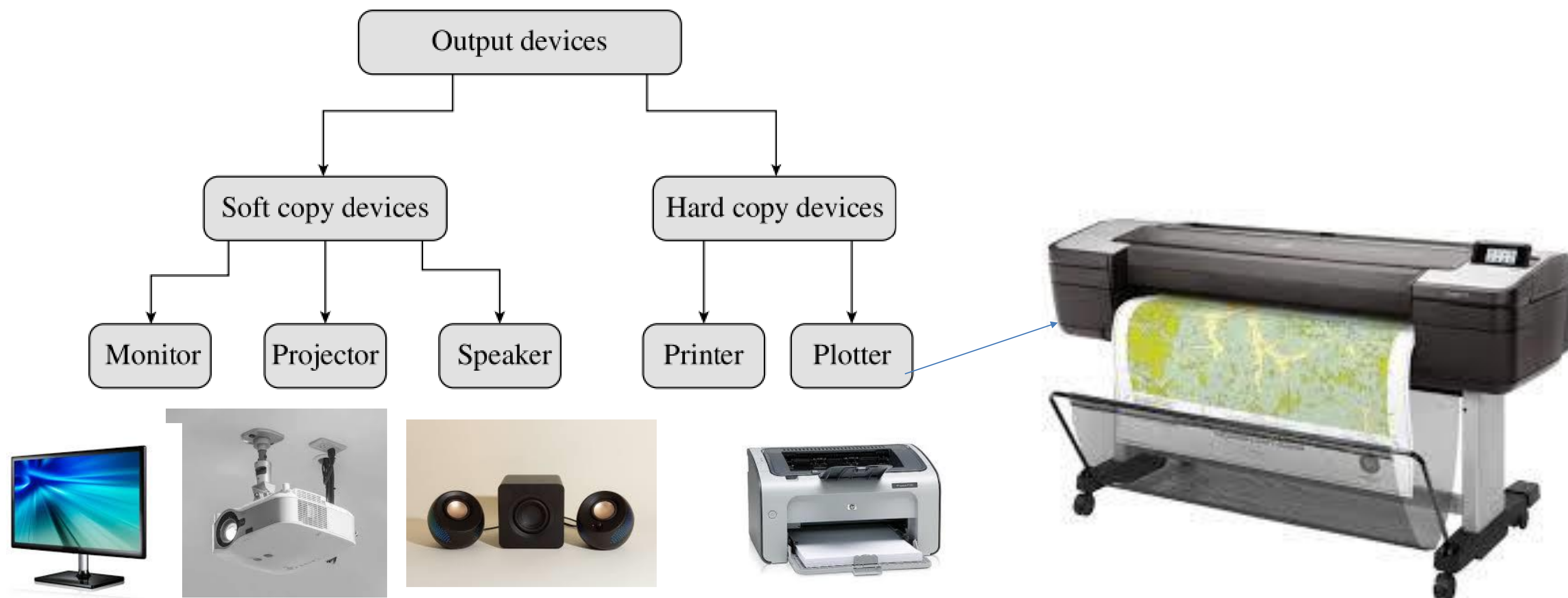
# Peripheral Devices/Input and Output Devices

- We need input and output devices for computers to be able to interact with its users
- There are different types of input/output devices,



**Input Devices** - An input device is used to feed data and instructions into the computer.

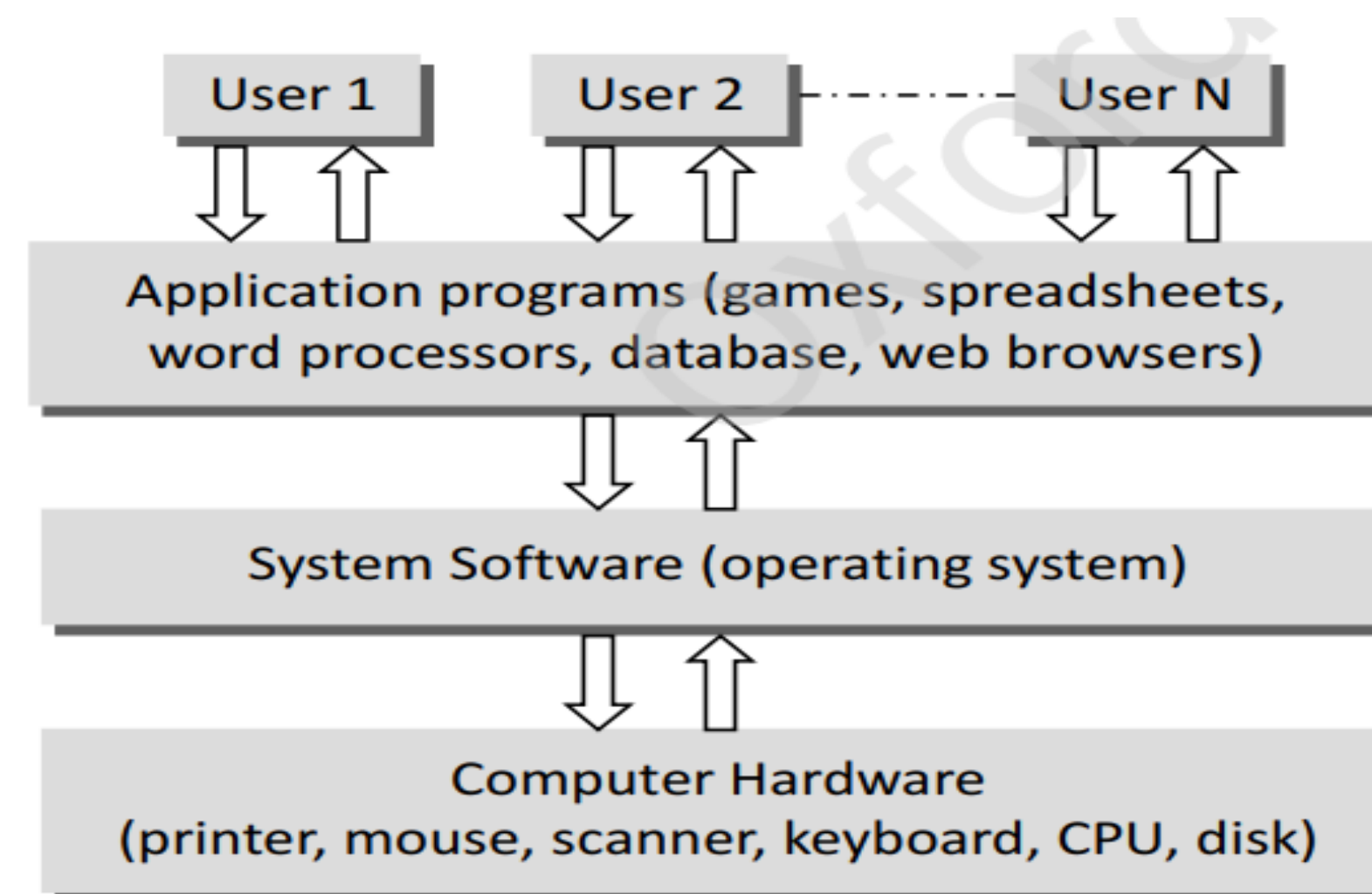
**Output Devices** - Any device that receives and display an information obtained after data processing by CPU is called as an output device.



**2. Computer Software:** Computer software can be broadly classified into two groups: **system software** and **application software**.

**System software** : provides a general programming environment in which programmers can create specific applications to suit their needs. This environment provides new functions that are not available at the hardware level and performs tasks related to execution of application programs.

**Application software** is designed to solve a particular problem for users. Simply put application software represents programs that allow users to do something besides simply running the hardware



## System Software :

System software is software designed to operate the computer hardware and to provide and maintain a platform for running application software.

The most widely used system software are:

- 1. Computer BIOS and Device Drivers :** BIOS is stored on a ROM chip built into the system. The key role of BIOS is to load and start the operating system. the code in the BIOS chip runs a series of tests called POST (Power On Self Test) to ensure that the system devices are working correctly. BIOS then locates software held on a peripheral device such as a hard disk or a CD, and loads and executes that software, giving it control of the computer. This process is known as booting

To summarize, BIOS performs the following functions:

1. Initializes the system hardware
2. Initializes system registers
3. Initializes power management system
4. Tests RAM
5. Tests all the serial and parallel ports
6. Initializes CD/DVD drive and hard disk controllers
7. Displays system summary information

## 2. Operating System:

- The primary goal of an operating system is to make the computer system (or any other device in which it is installed like a cell phone) convenient and efficient to use.
- An operating system offers generic services to support user applications.
- An operating system ensures that the system resources (such as CPU, memory, I/O devices) are utilized efficiently
- it is important for an operating system to have a control policy and algorithm to allocate the system resources.

### 3.Utility Software:

Utility software is used to analyse, configure, optimize, and maintain the computer system.

Utility programs may be requested by application programs during their execution for multiple purposes.

Some of them are as follows:

Disk defragmenters, Disk checkers, Disk cleaners, Disk space analysers, Disk partitions utilities, Backup utilities, Disk compression utilities, File managers, System profilers, Anti-virus utilities , Data compression utilities, Cryptographic utilities, Launcher applications, Registry cleaners, Network utilities, Command line interface (CLI) and Graphical user interface (GUI)3.

## 4. Compiler, Interpreter, Linker, and Loader

**Compiler:** It is a special type of program that transforms the source code written in a programming language (the source language) into machine language comprising just two digits, 1s and 0s (the target language). The resultant code in 1s and 0s is known as the object code. The object code is the one which will be used to create an executable program.

**Interpreter:** The interpreter, on the other hand, translates the instructions into an intermediate form, which it then executes. interpreter interprets the source code line by line

**Linker**(link editor binder) It is a program that combines object modules to form an executable program. The compiler automatically invokes the linker as the last step in compiling a program, to put together all the submodules object codes together.

**Loader** It is a special type of program that copies programs from a storage device to main memory, where they can be executed.

## **Application Software:**

Application software is a type of computer software that employs the capabilities of a computer directly to perform a user-defined task.

Typical examples of software applications are word processors, spreadsheets, media players, education software, CAD, CAM, data communication software, and statistical and operational research software

## PROGRAMMING LANGUAGES

- A programming language is a language specifically designed to express computations that can be performed by the computer.
- Programming languages are used to create programs that control the behaviour of a system, to express algorithms, or as a mode of human–computer communication.
- programming languages have a vocabulary of syntax and semantics for instructing a computer to perform specific tasks.
- The term programming language usually refers to high-level languages, such as BASIC, C, C++, COBOL, FORTRAN

- While high-level programming languages are easy for humans to read and understand, the computer actually understands the machine language that consists of numbers only.
- Each type of CPU has its own unique machine language
- In between the machine languages and high-level languages, there is another type of language known as assembly language.
- Assembly languages are similar to machine languages, but they are much easier to program because they allow a programmer to substitute names for numbers.
- There are two ways to convert the program written using any programming language into machine language is: compile the program or interpret the program

The question of which language is the best depends on the following factors:

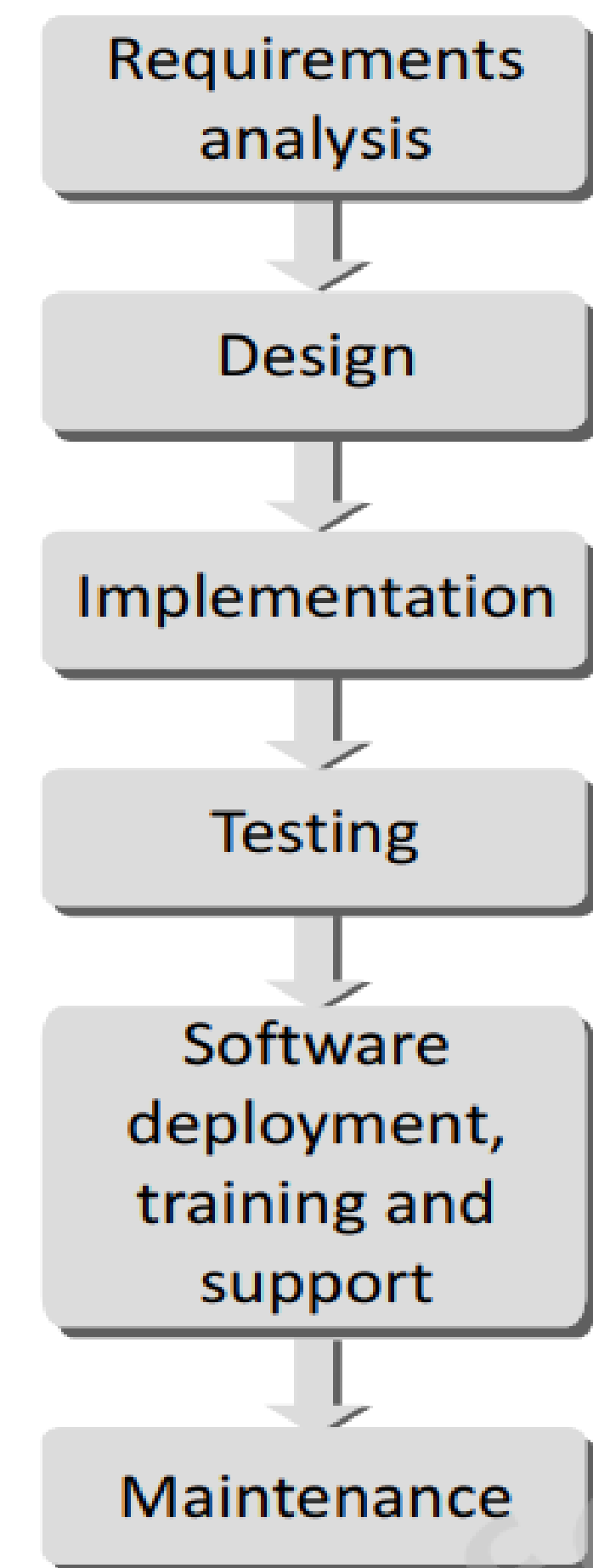
- The type of computer on which the program has to be executed
- The type of program
- The expertise of the programmer

## DESIGN AND IMPLEMENTATION OF EFFICIENT PROGRAMS

The entire program or software (collection of programs) development process is divided into a number of phases where each phase performs a well-defined task.

Moreover, the output of one phase provides the input for its subsequent phase.

**Requirements Analysis:** All the gathered users' requirements are analysed to pen down the scope or the objective of the overall software product. Documenting every identified requirement of the users in order to avoid any doubts or uncertainty regarding the functionality of the programs. The functionality, capability, performance, availability of hardware and software components are all analysed in this phase



## Design:

- A plan of actions is made before the actual development process could start.
- This plan will be followed throughout the development process.
- In the design phase the core structure of the software/program is broken down into modules.
- The solution of the program is then specified for each module in the form of algorithms, flowcharts, or pseudocodes.
- The design phase, therefore, specifies how the program/software will be built.

## Implementation:

- The designed algorithms are converted into program code using any of the high level languages.
- The particular choice of language will depend on the type of program like whether it is a system or an application program.
- The program codes are tested by the programmer to ensure their correctness. This phase is also called construction or code generation phase as the code of the software is generated in this phase.
- While constructing the code, the development team checks whether the software is compatible with the available hardware and other software components that were mentioned in the Requirements Specification Document created in the first phase.

## Testing

- In this phase, all the modules are tested together to ensure that the overall system works well as a whole product.
- Integration testing is done in this phase.
- Testing is done to ensure that the software is working as expected by the users' requirements that were identified in the requirements analysis phase.

## Software Deployment, Training, and Support

- After the code is tested and the software or the program has been approved by the users, it is then installed or deployed in the production environment.
- Software Training and Support is a crucial phase which is often ignored by most of the developers to use and fix up small errors

## Maintenance

- Maintenance and enhancements are ongoing activities which are done to cope with newly discovered problems or new requirements.
- Such activities may take a long time to complete as the requirement may call for addition of new code that does not fit the original design or an extra piece of code required to fix an unforeseen problem.
- As a general rule, if the cost of the maintenance phase exceeds 25% of the prior-phases cost then it clearly indicates that the overall quality of at least one prior phase is poor.
- In such cases, it is better to re-build the software (or some modules) before maintenance cost is out of control.

## PROGRAM DESIGN TOOLS: ALGORITHMS, FLOWCHARTS, PSEUDOCODES

1 **Algorithms:** In programming, algorithm is a **set of well-defined instructions in sequence** to solve the problem and it **provides a blueprint to write a program** to solve a given problem.

Algorithms are mainly used to,

- achieve **software reuse**.
- **reimplement** in any high-level language, such as C, C++, Java, and so on

In order to qualify as an algorithm, it should

- Be **precise**
- Be **unambiguous**
- Not even a single instruction must be repeated infinitely
- After the algorithm gets terminated, the **desired result** must be obtained

## Control Structures used in Algorithms

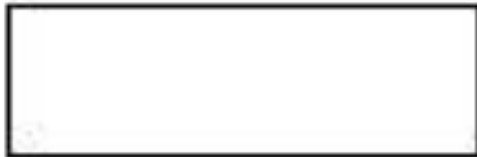
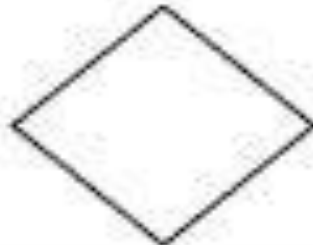
Broadly speaking, an algorithm can employ any of the three control structures, namely,

1. **Sequence:** Sequence means that each step of the algorithm is executed in the specified order
2. **Decision:** Decision statements are used when the outcome of the process depends on some condition.
3. **Repetition:** which involves executing one or more steps for a number of times, can be implemented using constructs such as while, do-while, and for loops.

- Selecting the Most Efficient Algorithm Many a times, you may formulate more than one algorithm for a problem. In such cases, you must always analyze all the alternatives and try to choose the most efficient algorithm. This can be done by analyzing **time** and **space** complexity of the algorithms designed.
- The **time complexity** of an algorithm is basically the running time of a program as a function of the input size. Similarly, the **space complexity** of an algorithm is the amount of computer memory that is required during the program execution as a function of the input size.

## 2 Flowcharts

- A flowchart is a graphical or symbolic representation of a process.
- It is basically used to design and document virtually complex processes to help the viewers to visualize the logic of the process, so that they can gain a better understanding of the process.
- When designing a flowchart, each step in the process is depicted by a different symbol and is associated with a short description.
- The symbols in the flowchart are linked together with arrows to show the flow of logic in the process.

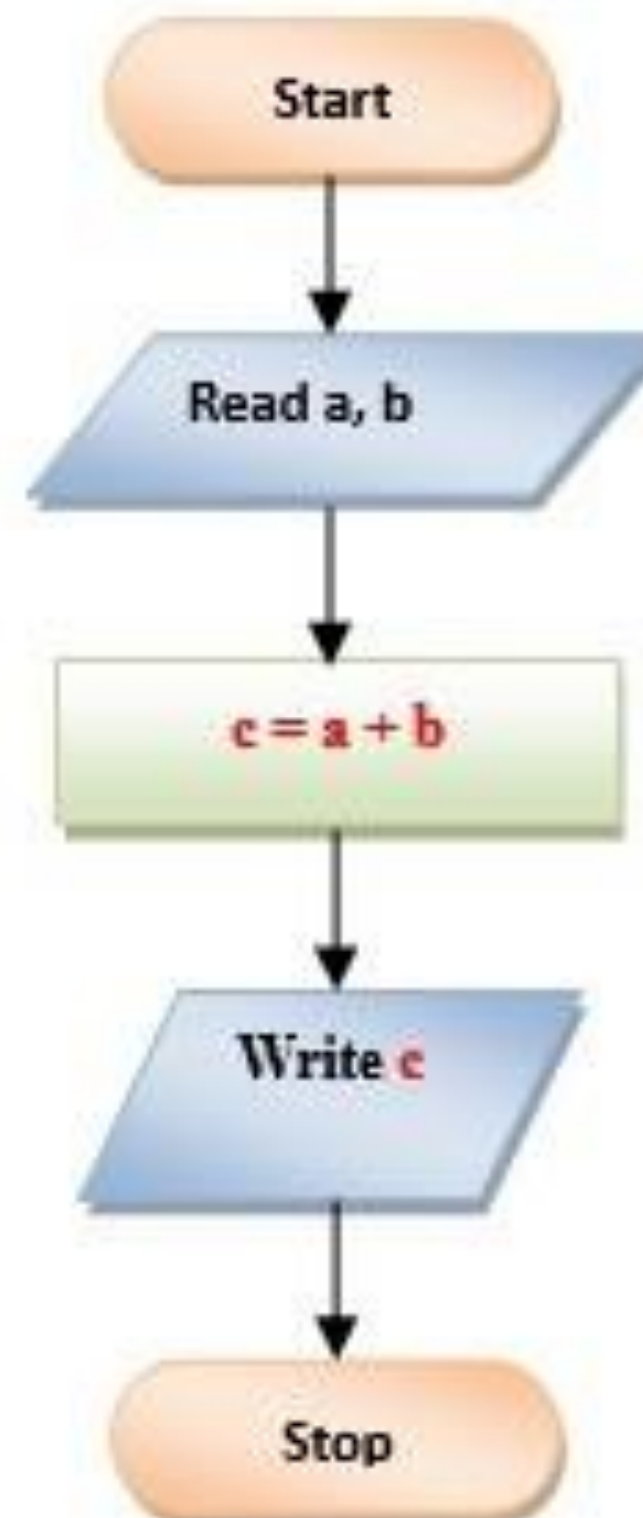
| Symbol                                                                               | Name               | Function                                                                                                     |
|--------------------------------------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------|
|    | Process            | Indicates any type of internal operation inside the Processor or Memory                                      |
|    | input/output       | Used for any Input / Output (I/O) operation. Indicates that the computer is to obtain data or output results |
|    | Decision           | Used to ask a question that can be answered in a binary format (Yes/No, True/False)                          |
|   | Connector          | Allows the flowchart to be drawn without intersecting lines or without a reverse flow.                       |
|  | Predefined Process | Used to invoke a subroutine or an Interrupt program.                                                         |
|  | Terminal           | Indicates the starting or ending of the program, process, or interrupt program                               |
|  | Flow Lines         | Shows direction of flow.                                                                                     |

## To find sum of two numbers

### Algorithm

1. Start
2. Read a, b
3.  $c = a + b$
4. Print or display c
5. Stop

### Flowchart



### Program

```

#include<stdio.h>

int main()
{
    int a, b, c;

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);
    c = a+b;

    printf("Sum of given two numbers is: %d", c);

    return 0;
}
  
```

## Finding Area of the rectangle

### Algorithm

1. Start
2. Read side length, a
3. Read side length b
4.  $area = a * b$
5. Print or display  $area$
6. Stop

### Flowchart



### Program

```

#include<stdio.h>

int main()
{
    int a, b, area;
    printf("Enter side length a: \n");
    scanf("%d", &a);

    printf("Enter side length b: \n");
    scanf("%d", &b);

    area = a*b;

    printf("Area of rectangle is: %d ", area);

    return 0;
}
  
```

## Finding Area of the square

### Algorithm

1. Start
2. Read length, L
3. **area** = L\*L
4. Print or display **area**
5. Stop

### Flowchart



### Program

```

#include<stdio.h>

int main()
{
    int L, area;

    printf("Enter length of square L: ");
    scanf("%d", &L);

    area = L*L;

    printf("Area of square is: %d", area);

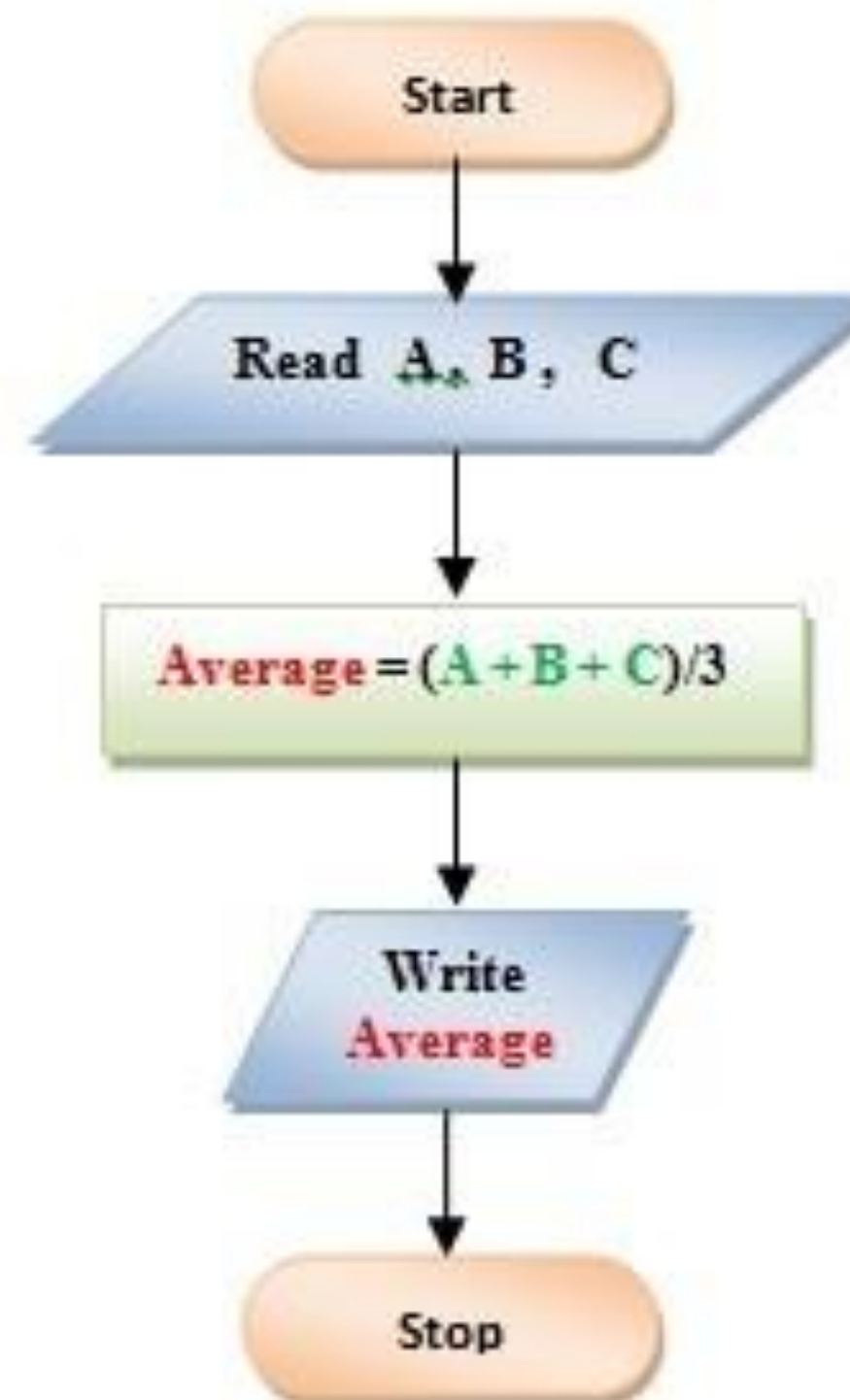
    return 0;
}
  
```

## Calculating the average for 3 numbers

### Algorithm

1. Start
2. Read 3 numbers A, B, C
3. Calculate the average by the equation:  
 $\text{Average} = (A + B + C)/3$
4. Print average
5. Stop

### Flowchart



### Program

```

#include<stdio.h>

int main()
{
    int A, B, C;
    float Average;

    printf("Enter values of A, B, C: \n");
    scanf("%d %d %d", &A, &B, &C);

    Average = (A+B+C)/3;

    printf("Average of given 3 numbers is: %f", Average);

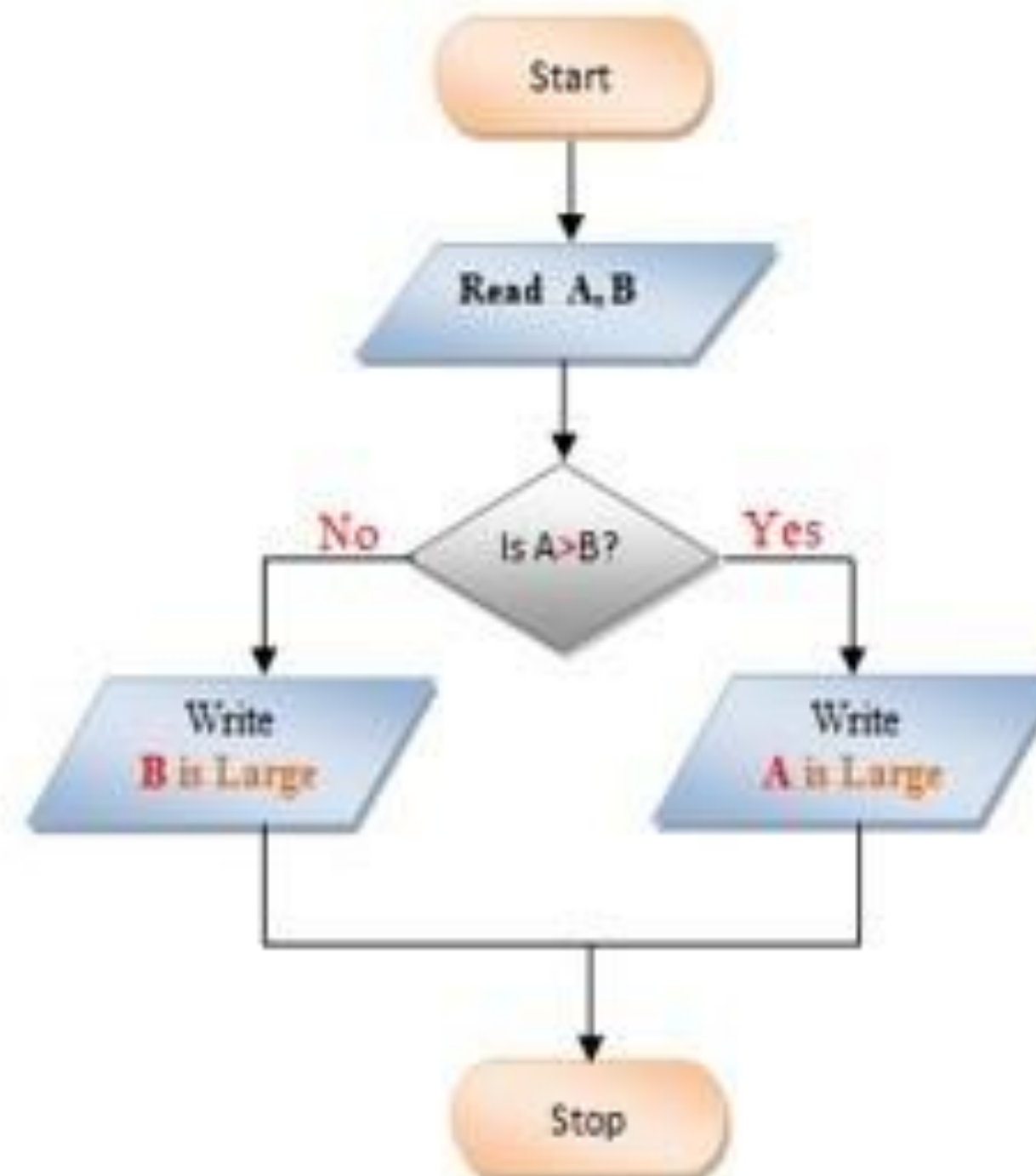
    return 0;
}
  
```

## Greatest of two numbers

### Algorithm

1. Start
2. Read A, B
3. If  $A > B$  then  
    Print A is large  
    else  
    Print B is large
4. Stop

### Flowchart



### Program

```

#include<stdio.h>

int main()
{
    int A, B;

    printf("Enter values of A, B: ");
    scanf("%d %d", &A, &B);

    if (A > B)
        printf("A is Larger");
    else
        printf("B is Larger");

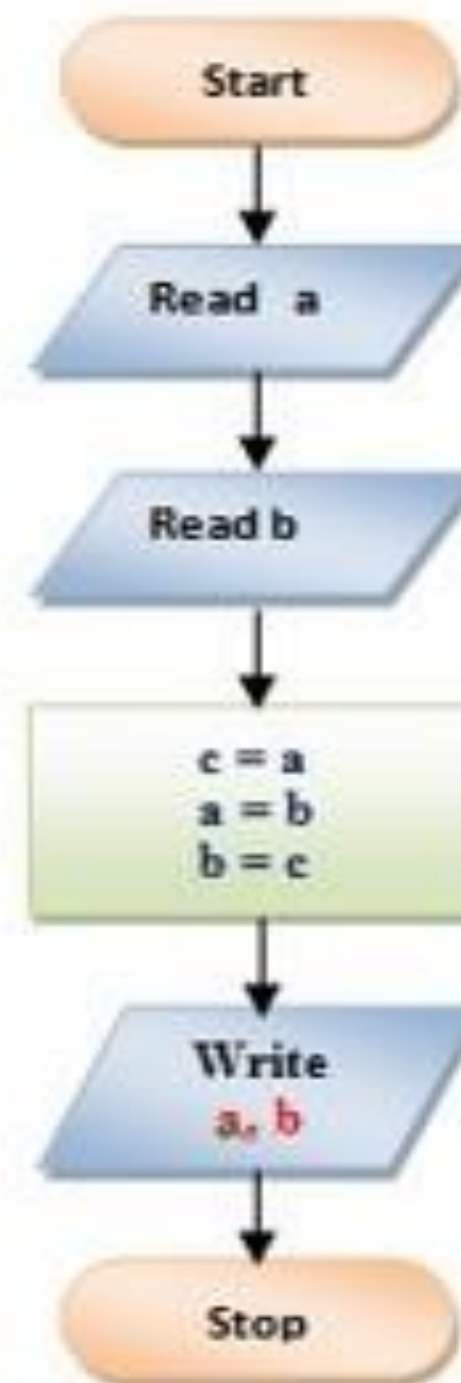
    return 0;
}
  
```

## Interchange the value of two numbers

### Algorithm

1. Start
2. Read two values into two variables a, b
3. Declare third variable, c  
 $c = a$   
 $a = b$   
 $b = c$
4. Print or display a, b
5. Stop

### Flowchart



### Program

```

#include<stdio.h>

int main()
{
    int a, b, c;
    printf("Enter value of a:");
    scanf("%d", &a);

    printf("Enter value of b:");
    scanf("%d", &b);

    c = a;
    a = b;
    b = c;

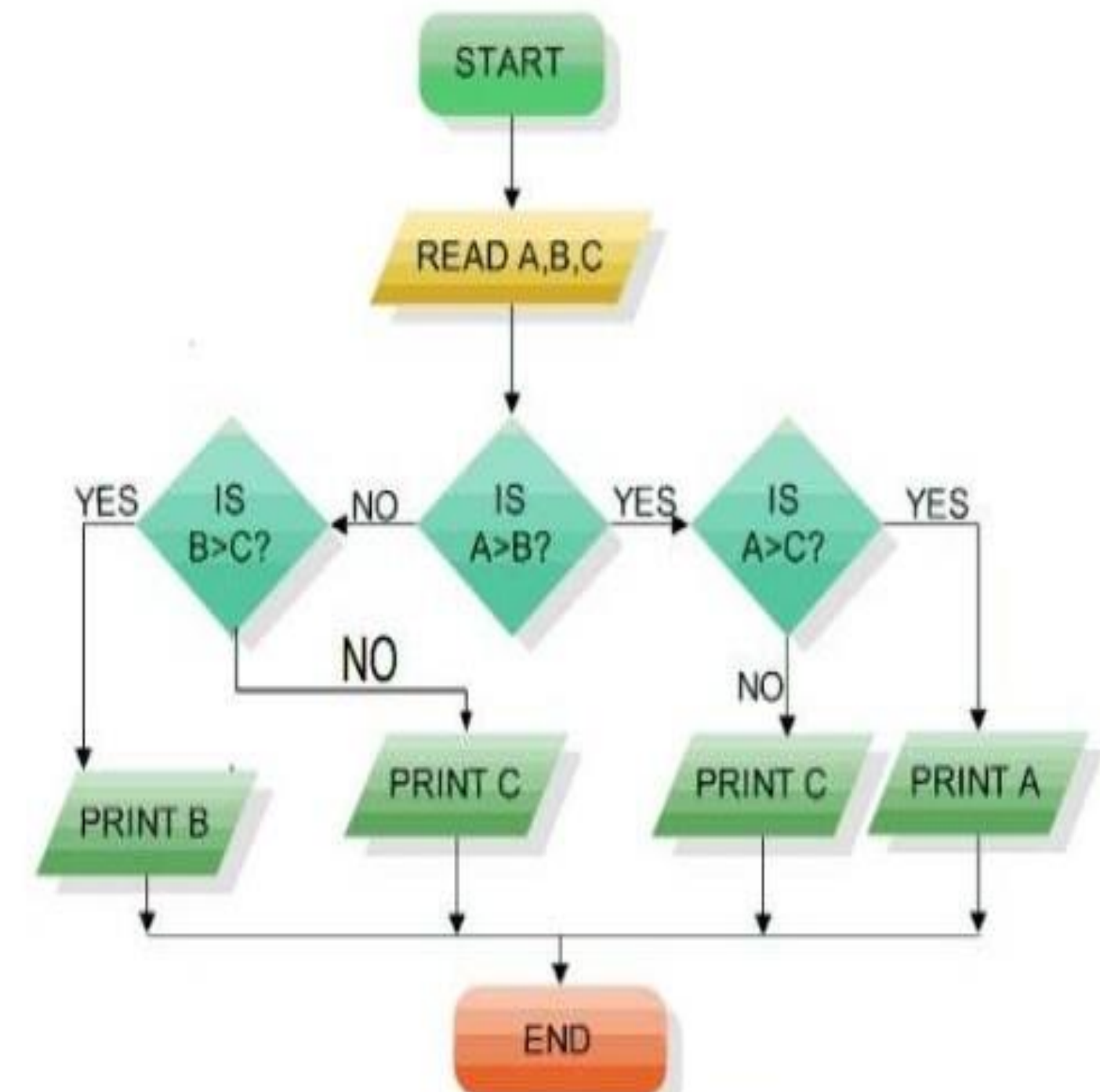
    printf("Values of a & b after swapping: ");
    printf("a = %d\n", a);
    printf("b = %d", b);

    return 0;
}
  
```

**Write an algorithm to find the largest among three different numbers entered by user.**

- **Step 1:** Start
- **Step 2:** Declare variables a,b and c.
- **Step 3:** Read variables a,b and c.
- **Step 4:** If  $a > b$ 
  - If  $a > c$   
Display a is the largest number.
  - Else  
Display c is the largest number.
- Else  
If  $b > c$   
Display b is the largest number.
- Else  
Display c is the greatest number.
- **Step 5:** Stop

**Flowchart – Find the Largest of Three Numbers**



# Sum of array of numbers

Step 1: Start

Step 2: Read a list of N numbers from the user as L

Step 3: Set  $\text{sum} = 0$ ,  $I = 0$

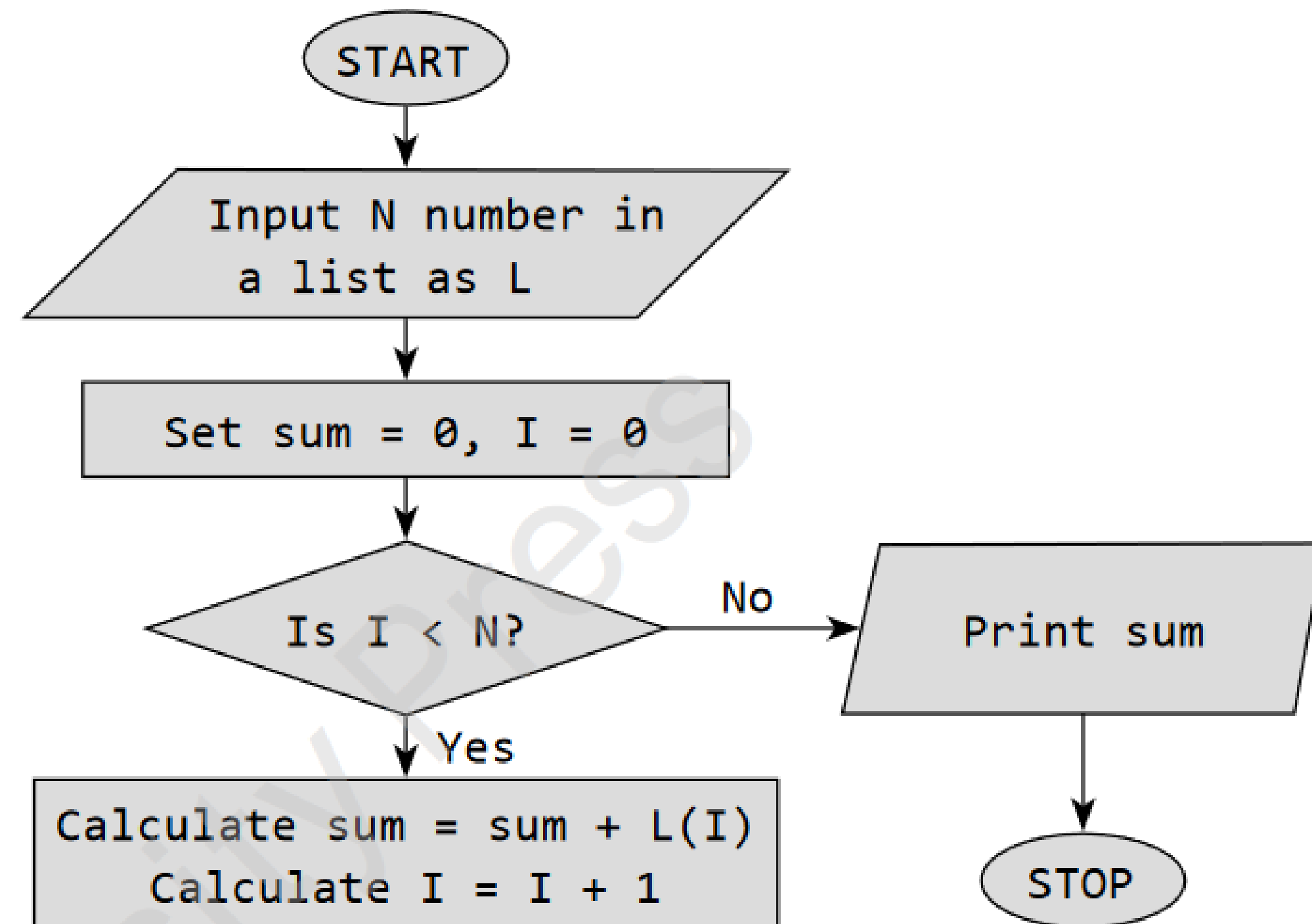
Step 4: WHILE  $I < N$

    Calculate  $\text{sum} = \text{sum} + L[I]$

    Calculate  $I = I + 1$

Step 5: Print sum

Step 6: End



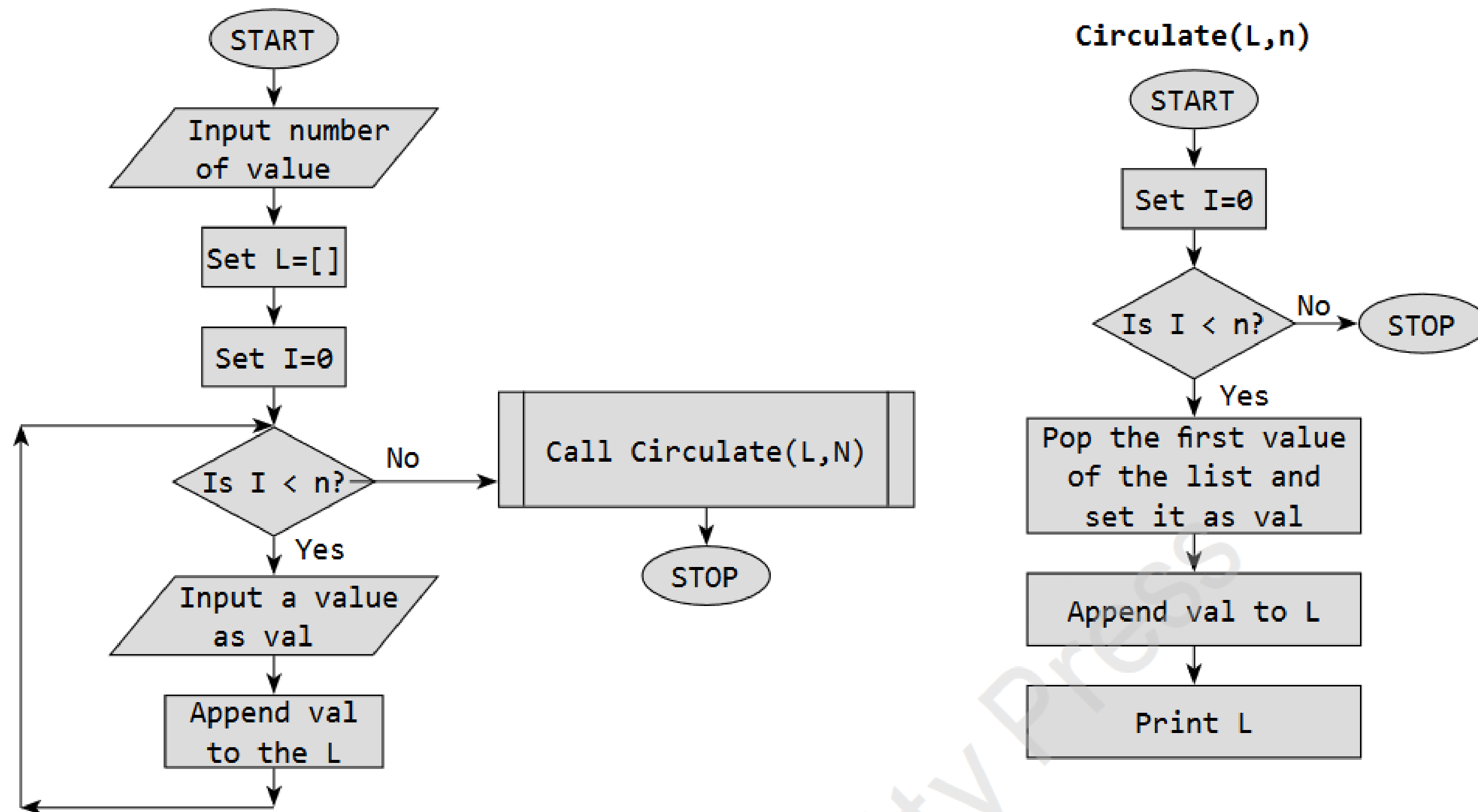
## Algorithm 2

Step 1: Start  
Step 2: Enter the number of elements in the list as  $n$   
Step 3: SET  $I = 0$   
Step 4: WHILE  $I < n$   
    Read an element  
    Append the element to the list  
    Print the list  
    Calculate  $I = I + 1$   
Step 5: CALL CIRCULATE (list,  $n$ )  
Step 6: End

CIRCULATE(list,  $n$ )

Step 1: Start  
Step 2: Set  $I = 0$   
Step 3: WHILE  $I < n$   
    Pop the first element from the list  
    Append it to the list (it now becomes the last element)  
    Print the list  
    Calculate  $I = I + 1$   
Step 4: End

## Flowchart 2



### Example 3 Test for Leap Year

#### Algorithm 3

Step 1: Start

Step 2: Enter a year as year

Step 3: Check IF((year%4==0 and year %100!=0)  
or (year%400 == 0)),

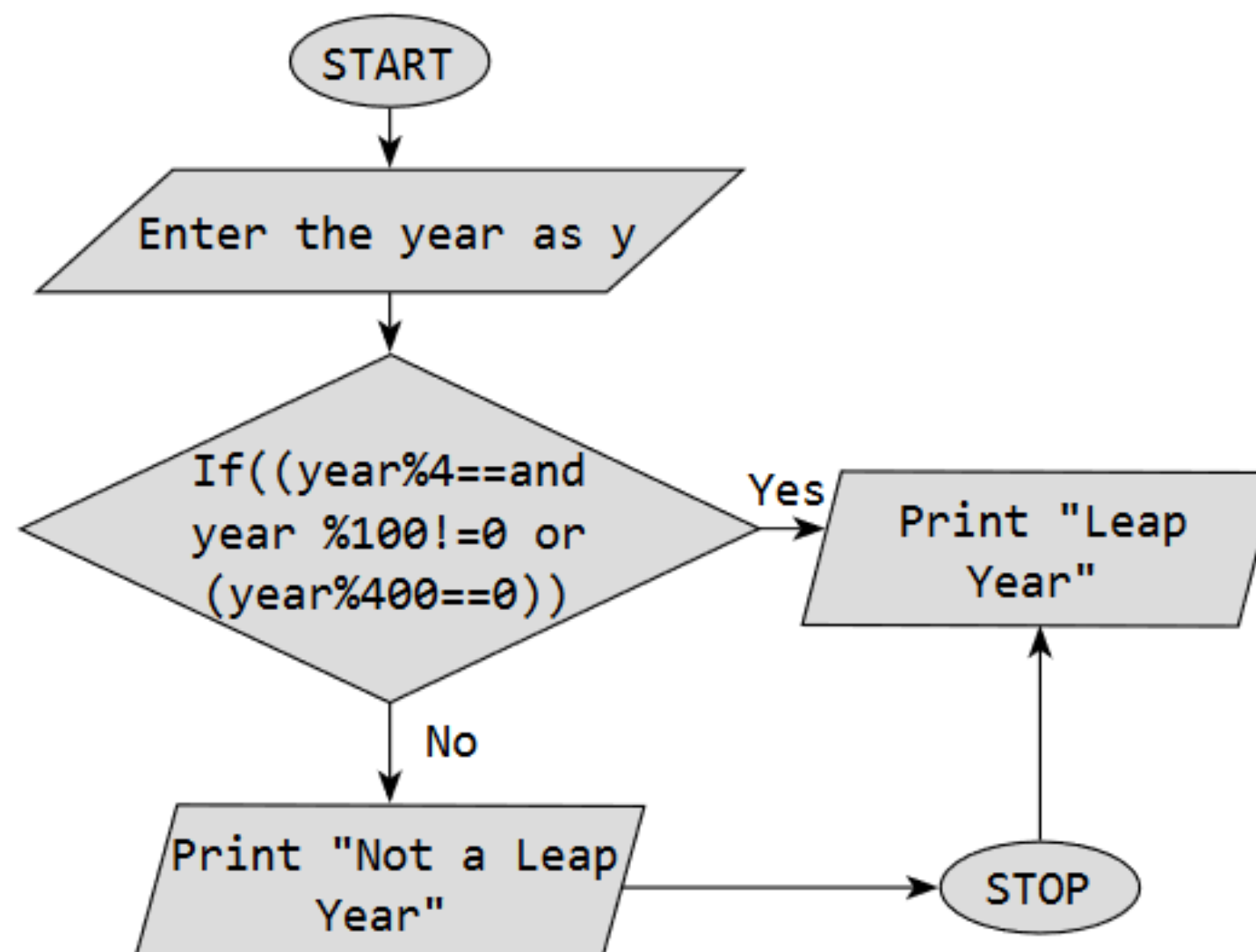
Then PRINT "Leap Year"

ELSE

PRINT "Not a Leap Year"

Step 4: End

#### Flowchart 3



### Example 4 Square root of a number

#### Algorithm 4

Step 1: Start

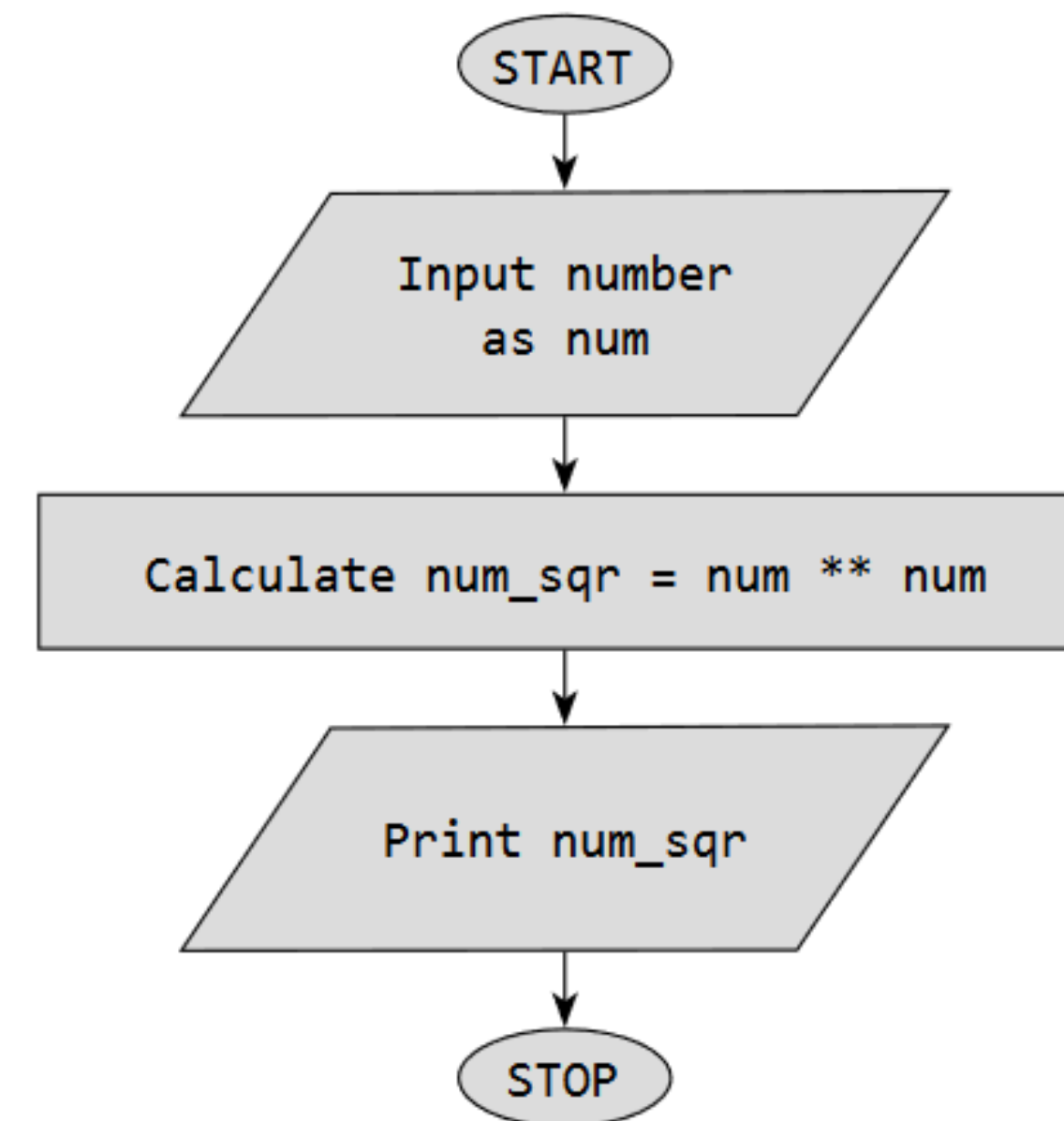
Step 2: Input the number as num

Step 3: Calculate square as num \*\* num

Step 4: Print square as calculated in Step 3

Step 5: End

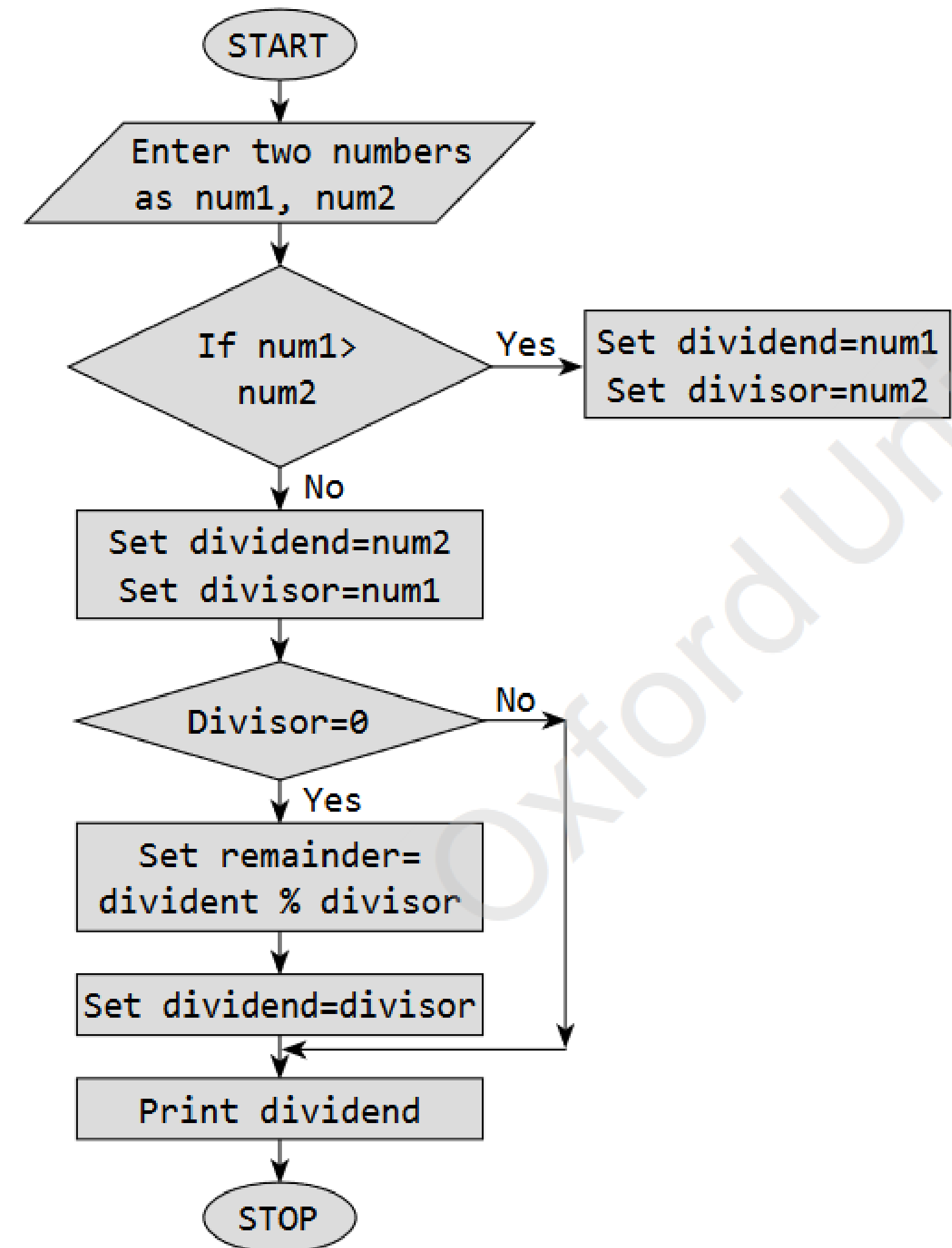
#### Flowchart 4



## Example 5 GCD of Two Numbers

### Algorithm 5

- Step 1: Start
- Step 2: Enter the two numbers as num1 and num2
- Step 3: Set larger of the two numbers as dividend
- Step 4: Set smaller of the two numbers as divisor
- Step 5: Repeat Steps 6-8 while divisor  $\neq 0$
- Step 6: Set remainder = dividend % divisor
- Step 7: Set dividend = divisor
- Step 8: Set divisor = remainder
- Step 9: Print dividend
- Step 10: End



## Example 6 Sum an Array of Numbers

### Algorithm 6

Step 1: Start

Step 2: Read a list of  $N$  numbers from the user  
as  $L$

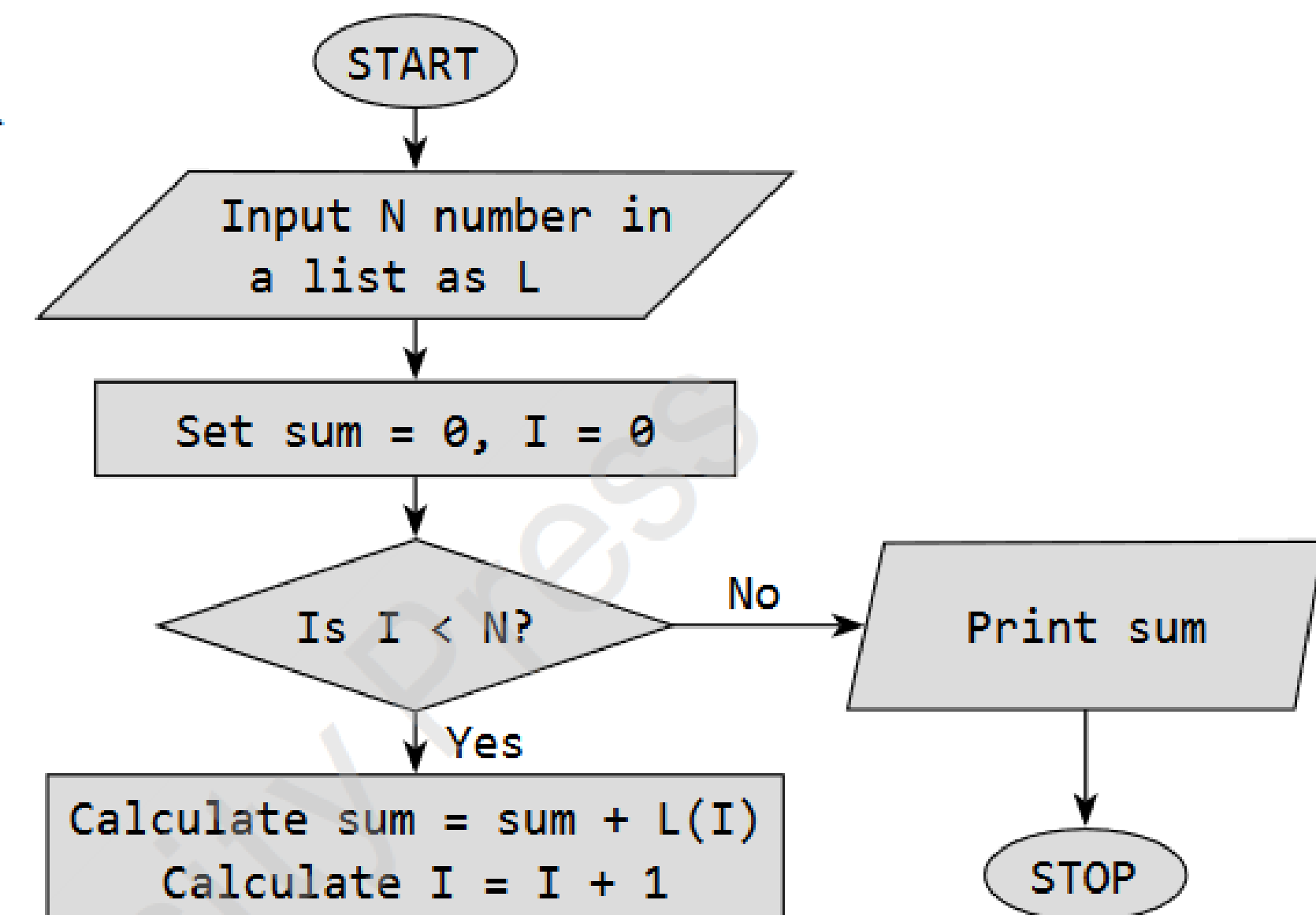
Step 3: Set  $\text{sum} = 0$ ,  $I = 0$

Step 4: WHILE  $I < N$   
    Calculate  $\text{sum} = \text{sum} + L[I]$   
    Calculate  $I = I + 1$

Step 5: Print sum

Step 6: End

### Flowchart 6



## Example 9 Finding Roots of Equations

### Algorithm 9

Step 1: Start

Step 2: Input three values as a, b and c

Step 3: Set  $D = (b \times b) - (4 \times a \times c)$

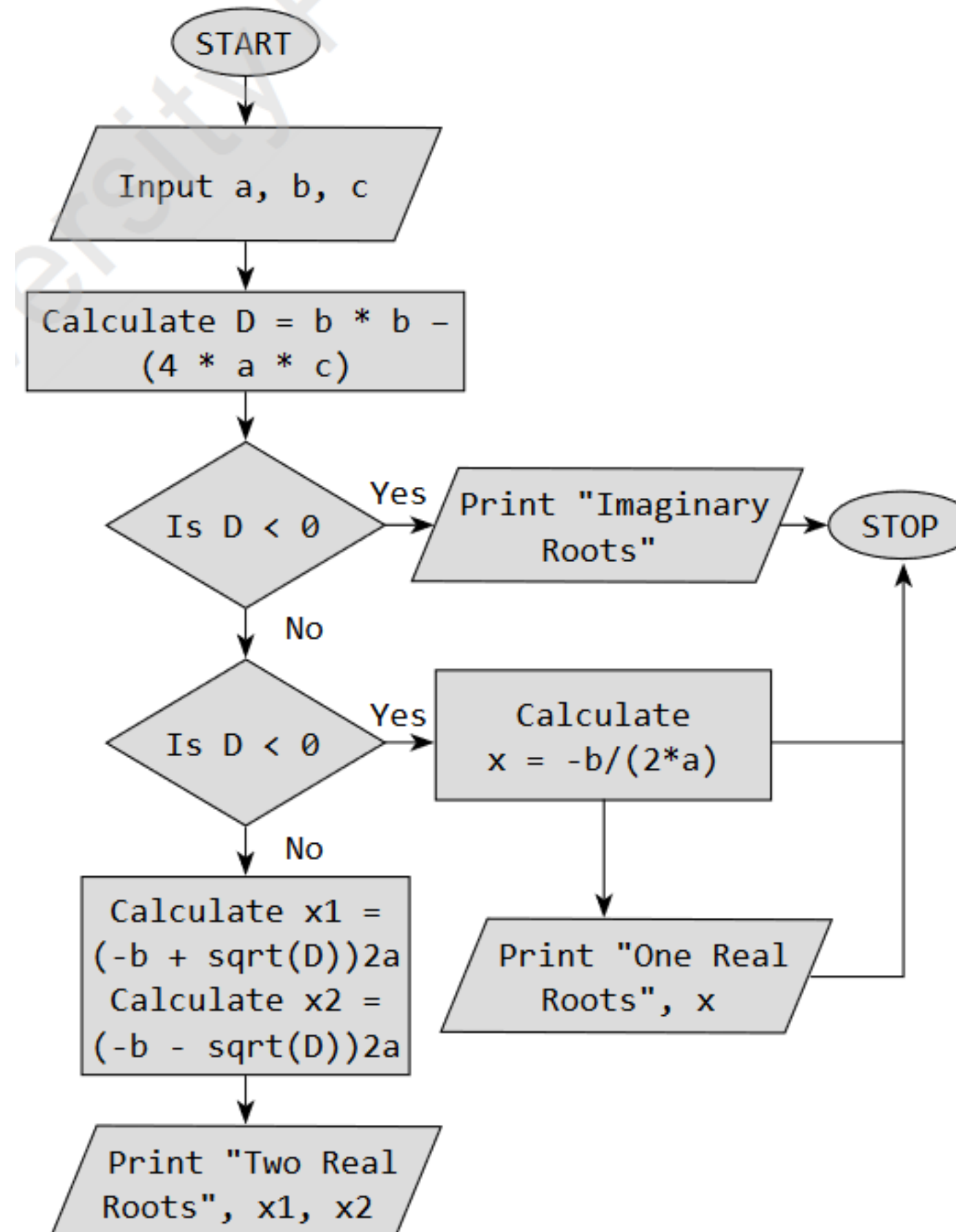
Step 4: If  $D < 0$ ,  
Print "Imaginary Roots".

Else If  $D = 0$ ,  
Set  $x = -b / (2a)$   
Print x

Else  
Set  $x_1 = (-b + \sqrt{D}) / 2a$   
Set  $x_2 = (-b - \sqrt{D}) / 2a$   
Print  $x_1, x_2$

Step 5: End

Flowchart 9



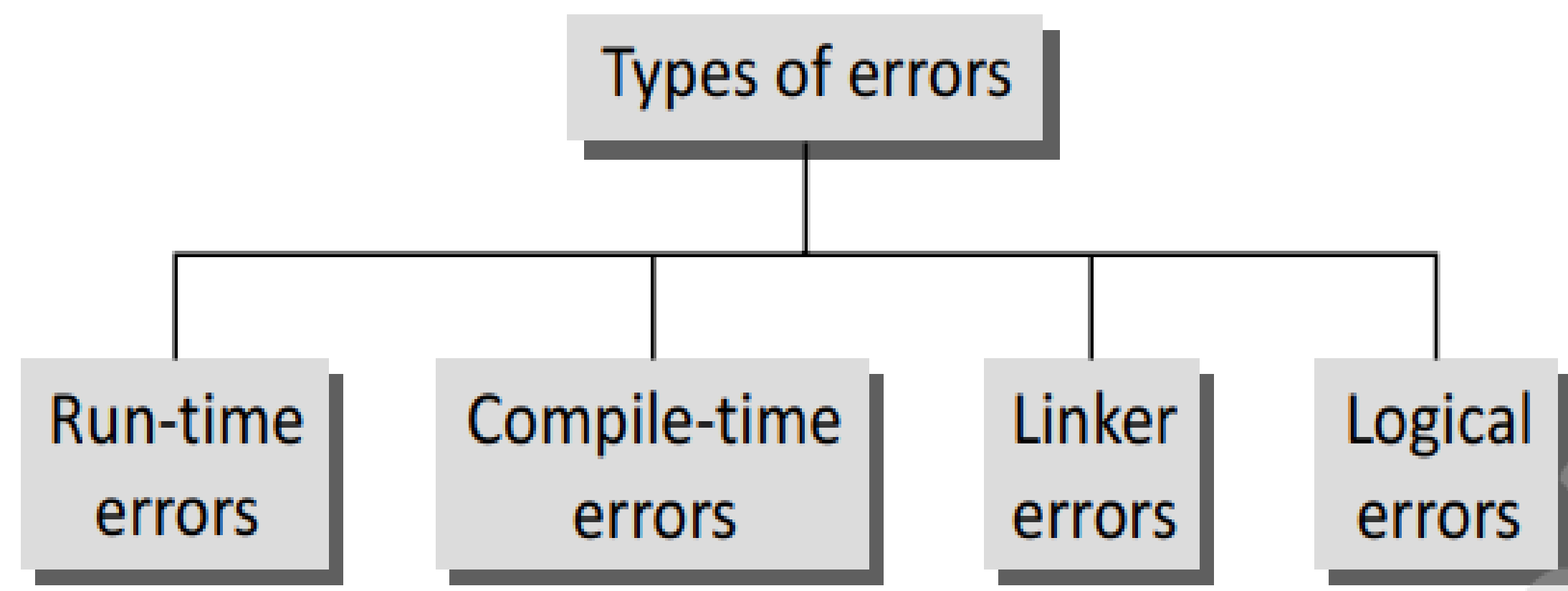
### 3 Pseudocodes

- Pseudocode is a compact and informal **high-level description of an algorithm** that uses the structural conventions of a programming language.
- Pseudocodes are an outline of a program that can be **easily converted into programming statements**.
- They consist of **short English phrases** that explain specific tasks within a program's algorithm. They **should not include keywords** in any specific computer language.
- The sole purpose of pseudocodes is to **enhance human understandability** of the solution.
- They are commonly used in textbooks and scientific publications for **documenting algorithms**, and for sketching out the program structure before the actual coding is done
- There are no standards defined for writing a pseudocode, because a pseudocode is **not an executable program**.

## TYPES OF ERRORS:

While writing programs, very often we get errors in our program. These errors if not removed will either give erroneous output or will not let the compiler to compile the program.

These errors are broadly classified under four groups:



## Run-time Errors

- As the name suggests, run-time errors occur when the program is being run executed. Such errors occur when the program performs some illegal operations like
  - Dividing a number by zero
  - Opening a file that already exists
  - Lack of free memory space
  - Finding square or logarithm of negative numbers

## Compile-time Errors

Again as the name implies, compile errors occur at the time of compilation of the program.

Such errors can be further classified as follows:

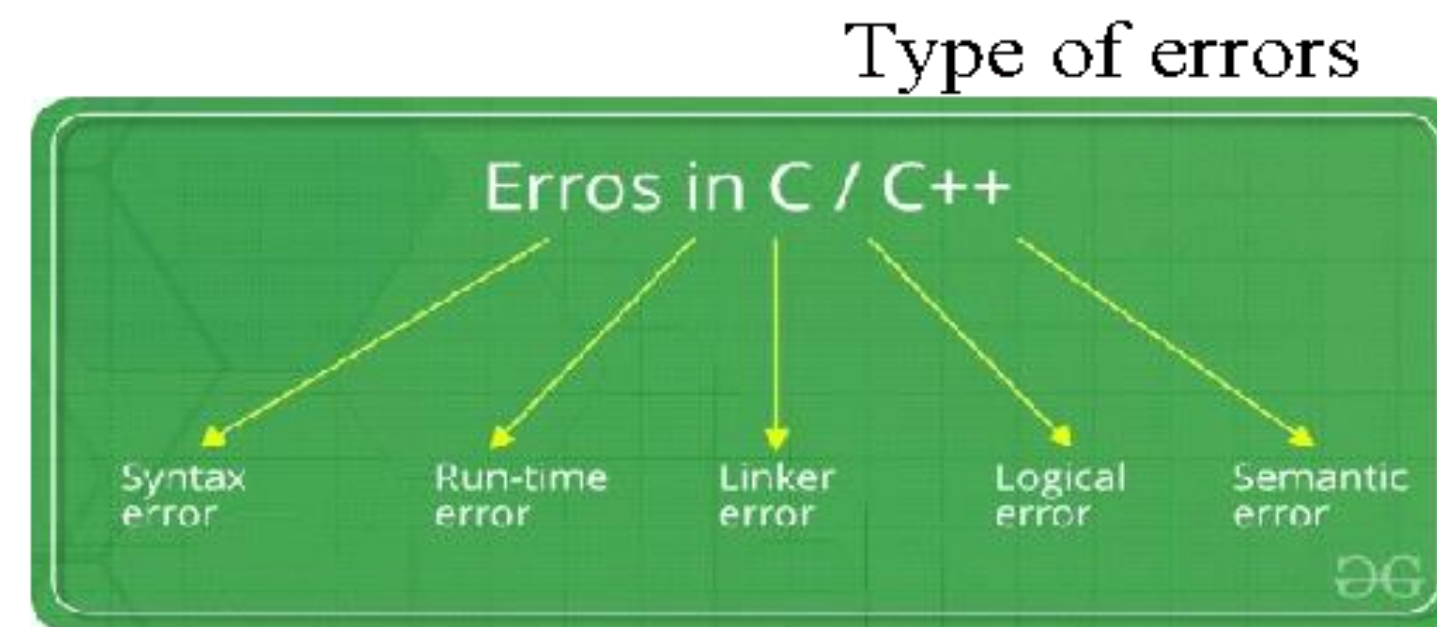
1. **Syntax Errors:** Syntax error is generated when rules of C programming language are violated.
2. **Semantic Errors:** Semantic errors are those errors which may comply with rules of the programming language but are not meaningful to the compiler
3. **Logical Errors:** Logical errors are errors in the program code that result in unexpected and undesirable output which is obviously not correct.
4. **Linker Errors:** These errors occur when the linker is not able to find the function definition for a given prototype.

## Errors in C/C++

Error is an illegal operation performed by the user which results in abnormal working of the program.

Programming errors often remain undetected until the program is compiled or executed. Some of the errors inhibit the program from getting compiled or executed. Thus errors should be removed before compiling and executing.

The most common errors can be broadly classified as follows.



1. **Syntax errors:** Errors that occur when you violate the rules of writing C/C++ syntax are known as syntax errors. This compiler error indicates something that must be fixed before the code can be compiled. All these errors are detected by compiler and thus are known as compile-time errors.

Most frequent syntax errors are:

- Missing Parenthesis (})
- Printing the value of variable without declaring it

Missing semicolon like this

```
// C program to illustrate syntax errora)
#include<stdio.h>void main()
{   int x = 10;int y = 15;
    printf("%d", (x, y)) // semicolon missed
}
```

Error:

error: expected ';' before '}' tokenb)

```
#include<stdio.h>int main(void)
{
    // while() cannot contain "." as an argument.while(.)
    {   printf("hello");
    }
    return 0;
}
```

Error:

error: expected expression before '.' tokenwhile(.)

2. Run-time Errors : Errors which occur during program execution(run-time) after successful compilation are called run-time errors. One of the most common run-time error is division by zero also known as Division error. These types of error are hard to find as the compiler doesn't point to the line at which the error occurs.

```
// C program to illustrate run-time error#include<stdio.h>
void main()
{   int n = 9, div = 0;
    // wrong logic
    // number is divided by 0,
    // so this program abnormally terminatesdiv = n/0;
    printf("resut = %d", div);
}
```

Error:

warning: division by zero [-Wdiv-by-zero]div = n/0;

Linker Errors: These error occurs when after compilation we link the different object files with main's object using Ctrl+F9 key(RUN). These are errors generated when the executable of the program cannot be generated. This may be due to wrong

function prototyping, incorrect header files. One of the most common linker error is writing Main() instead of main().

```
// C program to illustrate  
// linker error #include<stdio.h>  
void Main() // Here Main() should be main()  
{ int a = 10; printf("%d", a);  
}
```

Error:

(.text+0x20): undefined reference to `main'

**4. Logical Errors :** On compilation and execution of a program, desired output is not obtained when certain input values are given. These types of errors which provide incorrect output but appears to be error free are called logical errors. These are one of the most common errors done by beginners of programming.

These errors solely depend on the logical thinking of the programmer and are easy to detect if we follow the line of execution and determine why the program takes that path of execution.

```
// C program to illustrate logical error int main()  
{ int i = 0;  
  // logical error : a semicolon after loop for(i = 0; i < 3;  
  i++);  
  { printf("loop "); continue;  
  }  
  getchar(); return 0;  
}
```

No output

**5. Semantic errors :** This error occurs when the statements written in the program are not meaningful to the compiler.

```
// C program to illustrate semantic error void main()  
{ int a, b, c;  
  a + b = c; //semantic error  
}
```

Error

error: lvalue required as left operand of assignment a + b = c; //semantic error